

TRUSTFLOW

Money Movement Application

Hackathon Solution Blueprint

Core idea

A trust-first closed-loop wallet where users can send, request and collect simulated money - while the backend proves that money never disappears, duplicates, overspends or becomes inconsistent under retries and concurrent actions.

Built from two layers:

Layer	What it contains
Challenge-required layer	Send / request money, account balance, real-world reliability, backend, fake funds, scalable thinking.
Improvised innovation layer	Safe Send, duplicate shield, balanced ledger, Trust Lab, Smart Collect, explainable receipts and rule-based risk checks.

Source basis: PSTU IT Carnival 2026 - Hackathon Challenge: Money Movement Application (29 August 2026).

Document note: "REQUIRED" items come from the challenge PDF. "PROPOSED" and "BONUS" items are solution improvements.

Contents

1. Easy Bangla Summary
2. Challenge Requirements - Must Fulfill
3. Final Product Vision and Scope
4. Feature Blueprint and Priority
5. Core User Journeys
6. Financial Correctness Engine
7. Safe Send and Risk-Aware UX
8. Smart Collect and Money Requests
9. Trust Lab - Reliability Demonstration
10. System Architecture
11. Data Model and Ledger
12. API and Transaction State Design
13. Failure Scenarios and Expected Behavior
14. Security and Trust Controls
15. Scalability for a Growing Community
16. Recommended Hackathon Tech Stack
17. 9-Hour Build Plan
18. Demo Script and Pitch
19. Judge Q&A Preparation
20. Final Submission Checklist

1. Easy Bangla Summary

এই Hackathon-এর মূল কাজ হলো এমন একটি fake digital money system বানানো যেখানে একজন user অন্য user-কে টাকা send করতে পারবে, টাকা request করতে পারবে, এবং প্রতিটি registered user-এর একটি balance থাকবে। কিন্তু challenge-এর আসল গুরুত্ব UI নয়; টাকা movement যেন ভুল না হয়, duplicate না হয়, network retry বা একই সময়ে multiple action হলেও balance correct থাকে - সেটাই মূল বিষয়।

PDF অনুযায়ী এটি real bank integration নয়। Application-টিকে closed money ecosystem হিসেবে ধরা যাবে এবং registration-এর সময় প্রতিটি user-কে BDT 100,000 simulated balance দেওয়া যেতে পারে। Technology, framework, database এবং architecture বেছে নেওয়ার স্বাধীনতা আছে।

One-line solution

Build a mini digital wallet that is simple for users but engineered so that every money movement is atomic, traceable and safe under real-world failure cases.

আমাদের improvised version-এর নাম TrustFlow। এর visible innovation হবে Safe Send, Smart Collect এবং Trust Lab; আর backend-এর ভিতরে থাকবে atomic transfer, duplicate protection, concurrency protection এবং balanced ledger।

REQUIRED FROM CHALLENGE PDF

2. Challenge Requirements - Must Fulfill

Requirement	What the challenge expects	How TrustFlow fulfills it
Money movement	Users must be able to move / transfer money to another user.	Send Money flow with receiver verification, balance check, atomic transfer and receipt.
Money request	Users must be able to request money owed by another user.	Money Request with accept / reject, plus optional partial payment and purpose.
Account + balance	Every registered user has an account with a balance.	Wallet account; default simulated starting balance can be BDT 100,000.
Trustworthy behavior	Users expect every transaction to be correct, reliable and trustworthy.	Ledger, atomicity, idempotency, concurrency protection, audit trail.
Real-world conditions	Fast actions, network problems, simultaneous activity and unexpected input must be considered.	Duplicate shield, retries, DB transaction / row lock, validation and failure states.
Backend included	A working web or mobile app including its backend is expected.	Frontend + backend REST API + PostgreSQL.
Think beyond CRUD	Solution should demonstrate engineering thinking, not simple balance editing.	Dedicated Payment Engine and immutable ledger entries.

Scale awareness	Community may grow beyond 10 million users in 3 years.	Scalable data model, indexed queries, stateless API, modular architecture.
Technical freedom	No restriction on language, framework, database, architecture or development tools.	Use team-familiar stack; recommended stack given later.
Closed ecosystem	No need for real banks, cards, gateways or financial networks.	All funds are simulated inside the system.
Small but thoughtful	Build a focused working product rather than many incomplete features.	P0 reliability first; P1 innovations only after core is stable.

Important interpretation

The problem is not “how to make a Send button.” The problem is “how to guarantee that one intended transfer causes exactly one correct financial effect.”

PROPOSED IMPROVEMENT

3. Final Product Vision and Scope

3.1 Product positioning

TrustFlow is a **trust-first community wallet** for simulated peer-to-peer money movement. It keeps the original challenge scope, then adds a small number of innovations that directly improve safety, explainability and demo value.

Product pillar	Meaning
Move	Send money safely to another registered user.
Request	Collect money through requests, including optional partial payments.
Protect	Prevent duplicate deductions, overspending and concurrency bugs.
Explain	Show exactly what happened, when, why and how the balance changed.
Prove	Use Trust Lab to demonstrate failure resistance live.

3.2 Scope boundaries

- No real bank / card / MFS / payment gateway integration.
- No attempt to replicate the entire feature set of bKash, Nagad or a bank.
- No unnecessary microservices during the hackathon; use a modular monolith.
- No “AI fraud detection” claim without a real dataset and trained model.
- Every optional feature is lower priority than transaction correctness.

REQUIRED + PROPOSED MIX

4. Feature Blueprint and Priority

Priority	Feature	Type	Why it matters
P0	Register / Login	Required foundation	Creates the registered user accounts used for money movement.
P0	Wallet Balance	Required foundation	Shows available simulated balance.
P0	Send Money	Required	Primary challenge action.
P0	Request Money + Accept / Reject	Required	Second primary challenge action.
P0	Transaction History	Trust foundation	Makes money movement visible and traceable.
P0	Atomic Transfer	Engineering must-have	Either debit + credit both happen, or neither happens.
P0	Insufficient Balance Protection	Engineering must-have	Prevents negative balances.
P0	Duplicate Payment Shield	Proposed reliability	Prevents double deduction from double-click or retry.
P0	Concurrent Spend Protection	Proposed reliability	Prevents two simultaneous transfers from spending the same balance.
P1	Safe Send	Proposed UX innovation	Extra confirmation for new / risky transfers.
P1	Trust Receipt	Proposed UX innovation	Explains transaction ID, status and balance change.
P1	Trust Lab	Proposed demo innovation	Proves reliability live to judges.
P1	Smart Collect / Partial Pay	Proposed product innovation	Makes request-money more useful in real life.
P2	Group Collect	Bonus	Tracks multiple people paying one shared expense.
P2	Hold & Release	Bonus	Safer payment for community marketplace / services.
P3	Natural-language request creation	Bonus AI	Convenience only; never executes money movement without confirmation.

5. Core User Journeys

5.1 Send Money

1. Choose or search a registered receiver.
2. Show receiver identity card so the sender can verify the person.
3. Enter amount and optional purpose.
4. Validate amount, receiver, balance and duplicate request key.
5. Run risk rules; activate Safe Send only when needed.
6. Confirm the transaction.
7. Execute debit + credit atomically.
8. Show Trust Receipt with transaction ID and new balance.

```
Hasan balance: BDT 100,000
Send BDT 2,500 -> Saif
Result:
  Hasan = BDT 97,500
  Saif = BDT 102,500
  Ledger difference = BDT 0
```

5.2 Request Money

9. Requester selects payer and amount.
10. Optional purpose / note is attached.
11. Payer receives a pending request.
12. Payer can Accept, Reject or (proposed) Pay Partially.
13. On payment, the same Payment Engine executes the financial transfer.
14. Request status updates and both users receive history entries.

5.3 Registration and starting funds

The challenge explicitly allows automatically funding each newly registered user with **BDT 100,000** in simulated money. Treat that event as controlled system issuance rather than user-to-user transfer, so later system-total checks remain explainable.

CORE ENGINEERING

6. Financial Correctness Engine

Golden rule

A transfer must be all-or-nothing, one-time, balance-safe and traceable.

6.1 Atomicity

Sender debit and receiver credit happen inside the same database transaction. If any step fails, the entire operation rolls back.

```
BEGIN
  lock sender account
  validate balance
  create transaction
  debit sender
  credit receiver
  create ledger entries
COMMIT

If any step fails -> ROLLBACK
```

6.2 Idempotency / Duplicate Payment Shield

Every send request carries an **idempotency_key**. If the same key arrives again because of double-click or network retry, the backend returns the already-created result instead of charging again.

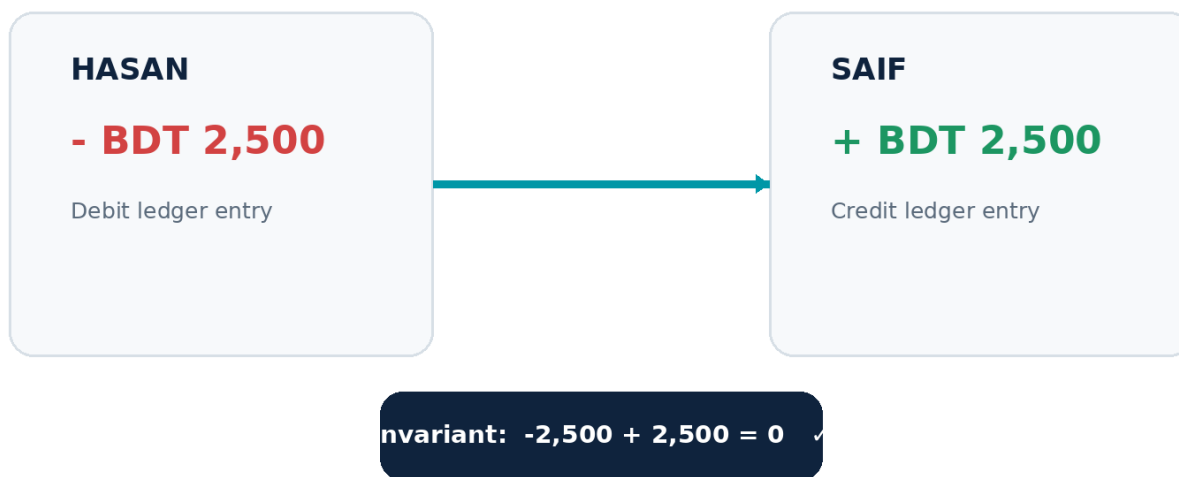
6.3 Concurrent spend protection

If BDT 1,000 is available and BDT 700 + BDT 700 are requested simultaneously, only one may succeed. Use a DB transaction with row locking or an equivalent serializable update so the second request sees the updated balance.

6.4 Balanced ledger

Balanced Ledger: money cannot silently disappear

Example: Hasan sends BDT 2,500 to Saif



For every user-to-user transfer, total ledger delta must equal zero. This gives the team a simple invariant to test and a powerful explanation to judges.

6.5 Money conservation check

User-to-user transfer invariant:
 $\text{SUM}(\text{all debits} + \text{all credits}) = 0$

System integrity check:
 $\text{Issued simulated funds} - \text{current wallet funds} - \text{reserved/held funds} = 0$

PROPOSED USER SAFETY INNOVATION

7. Safe Send and Risk-Aware UX

Safe Send does not slow every transfer. It applies extra friction only when simple explainable rules indicate higher mistake risk.

Signal	Example	Response
New recipient	First transfer to @hasan72	Show receiver identity + hold-to-confirm.
Large amount	Amount exceeds a chosen threshold	Show clear amount warning and second confirmation.
Rapid repeat	Same receiver + amount attempted again quickly	Treat as possible duplicate and stop / verify.
Many failed attempts	Repeated insufficient-balance or validation failures	Temporarily slow actions and show explanation.
Normal repeat transfer	Small amount to a known receiver	Keep fast one-step confirmation.

Why this is defensible

This is a transparent rule engine, not fake “AI fraud detection.” Every warning can be explained to the user and to the judges.

PROPOSED REQUEST-MONEY INNOVATION

8. Smart Collect and Money Requests

8.1 Purpose-aware request

Request from: Hasan
Amount: BDT 1,200
Purpose: Dinner
Status: Pending

8.2 Partial payment

If a BDT 5,000 request cannot be paid fully, the payer can optionally pay BDT 2,000. The request becomes PARTIALLY_PAID with BDT 3,000 remaining.

8.3 Group Collect (P2)

Trip Expense: BDT 4,000
Saif BDT 1,000 PAID
Hasan BDT 1,000 PENDING
Rahim BDT 1,000 PAID
Karim BDT 1,000 PENDING
Collected: BDT 2,000 / 4,000

8.4 Hold & Release (P2)

Optional community-marketplace mode: sender reserves the amount, then releases it after an agreed condition. Do not build this until P0 reliability is complete.

PROPOSED DEMO DIFFERENTIATOR

9. Trust Lab - Reliability Demonstration

Trust Lab - prove reliability live

A judge-facing demo screen that intentionally attacks the money engine.

Duplicate Click Test	5 API requests	1 financial effect	4 duplicates blocked
Concurrent Spend Test	Balance: BDT 1,000	BDT 700 + BDT 700	One succeeds; one rejected
Network Retry Test	First response times out	Client retries request	Same result; no re-deduction
Ledger Integrity Test	Debit = Credit	System difference = BDT	Ledger healthy

Trust Lab is an admin/demo-only screen. Its purpose is not production functionality; its purpose is to prove the engineering choices visually during judging.

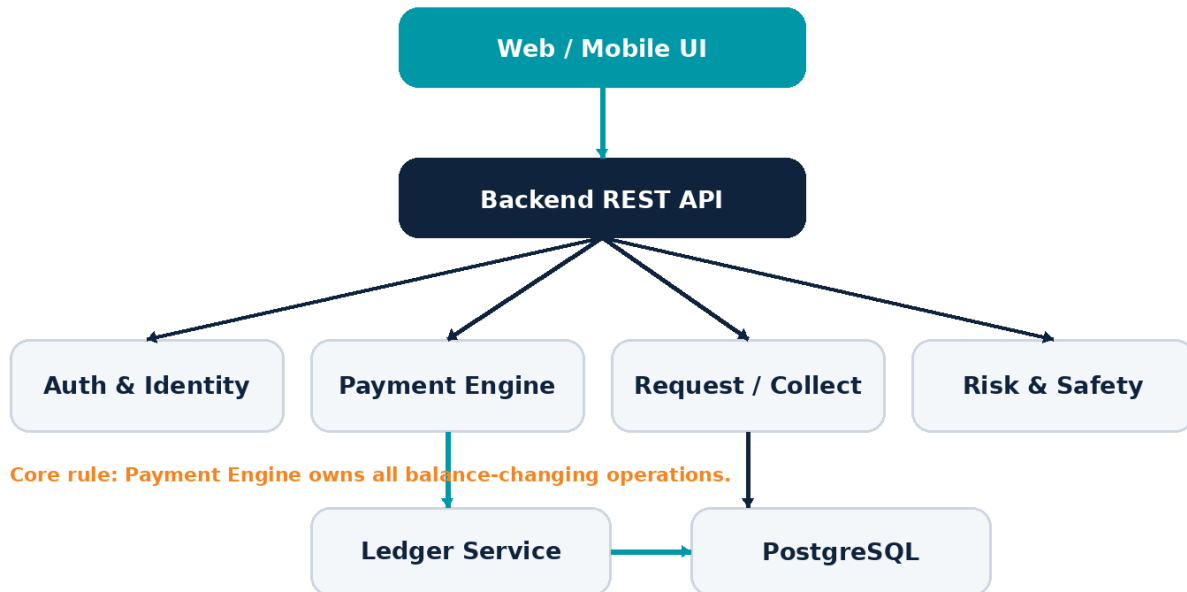
Test	Attack	Expected result
Duplicate Click	Send the same logical transaction 5 times.	1 transaction created; 4 duplicates return the same result.
Concurrent Spend	Attempt two BDT 700 transfers from a BDT 1,000 account at the same time.	One succeeds; one fails; final balance BDT 300.

Network Retry	Simulate lost response then retry with same key.	No second deduction; original result is returned.
Invalid Input	Try 0, negative, huge value, self-transfer or missing receiver.	No financial mutation; clear validation response.
Ledger Integrity	Reconcile debits and credits.	Difference BDT 0; status HEALTHY.

10. System Architecture

TrustFlow - Hackathon Architecture

Modular monolith: fast to build, strong enough to demonstrate financial correctness



- Frontend handles user interaction only; it never directly changes balances.
- Payment Engine is the only module allowed to perform balance-changing operations.
- Request / Collect module creates obligations, but uses Payment Engine when money actually moves.
- Risk module produces warnings / confirmation rules; it does not bypass financial correctness.
- Ledger module writes immutable financial entries for completed transfers.
- PostgreSQL provides ACID transactions, constraints, locking and indexed querying.

PROPOSED IMPLEMENTATION

11. Data Model and Ledger

Table	Important fields	Purpose
users	id, name, username, email, password_hash	Identity and authentication.
accounts	id, user_id, balance, status, version	Wallet state.
transactions	id, sender_account_id, receiver_account_id, amount, status, idempotency_key, purpose, created_at	One logical money movement.
ledger_entries	id, transaction_id, account_id, direction, amount, created_at	Immutable debit / credit audit trail.

money_requests	id, requester_id, payer_id, amount, paid_amount, status, purpose, created_at	Request lifecycle and partial payment tracking.
risk_events	id, transaction_id, rule_code, score, message	Explainable warning reasons.
audit_events	id, actor_id, event_type, entity_id, metadata, created_at	Security and important state-change trace.

11.1 Important database constraints

- amount > 0
- sender_account_id != receiver_account_id
- idempotency_key UNIQUE per sender or globally, depending on design
- Foreign keys for all account / user references
- Only valid enum states for transaction and request status
- Balance must not become negative through the Payment Engine
- Ledger entries are append-only after commit

PROPOSED IMPLEMENTATION

12. API and Transaction State Design

Method	Endpoint	Purpose
POST	/auth/register	Create user + account + simulated starting funds.
POST	/auth/login	Authenticate.
GET	/wallet	Current balance and summary.
POST	/transfers	Create a transfer using idempotency key.
GET	/transactions	Paginated transaction history.
POST	/requests	Create a money request.
POST	/requests/{id}/accept	Pay / accept request.
POST	/requests/{id}/reject	Reject request.
POST	/requests/{id}/partial-pay	Optional partial payment.
GET	/trust-lab/integrity	Demo reconciliation status.

12.1 Transaction state machine

CREATED -> PROCESSING -> COMPLETED
 \ -> FAILED

Safe Send variant:

CREATED -> PENDING_CONFIRMATION -> PROCESSING -> COMPLETED

12.2 Request state machine

PENDING -> PAID
 -> REJECTED
 -> CANCELLED
 -> PARTIALLY_PAID -> PAID

REQUIRED REAL-WORLD THINKING

13. Failure Scenarios and Expected Behavior

Scenario	Naive failure	TrustFlow expected behavior
Double-click Send	Two transfers happen.	Same idempotency key -> exactly one financial effect.
Response lost after success	User retries and pays twice.	Retry returns original transaction result.
Two simultaneous sends	Both see old balance and overspend.	Account lock / atomic update allows only valid spend.
Server error after debit	Money disappears before receiver credit.	DB rollback restores original state.
Negative / zero amount	Corrupt or nonsensical transaction.	Reject before any DB mutation.
Self-transfer	Meaningless ledger movement.	Reject.
Unknown receiver	Money could be misdirected.	Reject and show receiver-not-found.
Insufficient balance	Balance becomes negative.	Reject atomically.
Repeated request acceptance	Same request paid more than once.	State + idempotency prevent second payment.
History query at scale	Slow dashboard.	Indexed, paginated retrieval by account + created_at.

PROPOSED TRUST CONTROLS

14. Security and Trust Controls

- Hash passwords using a standard adaptive password hash (e.g., bcrypt / Argon2).
- Use authenticated sessions or signed access tokens; never trust user_id supplied by the client.
- Authorize every request: a user can only act on their own wallet / requests unless explicitly permitted.

- Validate all money amounts on the backend; frontend validation is only convenience.
- Use parameterized queries / ORM to prevent injection.
- Rate-limit login and sensitive transaction endpoints if time permits.
- Do not expose full emails or sensitive data in receiver identity cards.
- Log security-relevant events separately from user-visible transaction history.
- Never allow a normal API to directly set arbitrary account balances.

REQUIRED SCALE AWARENESS

15. Scalability for a Growing Community

The challenge mentions a community that could exceed 10 million users in three years. The hackathon prototype does not need 10 million users, but the design should avoid obvious dead ends.

Area	Prototype choice	Scale-aware direction
API	Stateless backend	Horizontal replicas behind a load balancer.
Database	Single PostgreSQL instance	Read replicas / partitioning only when actually needed.
Transaction lookup	Indexed account_id + created_at	Cursor pagination; avoid loading full history.
Idempotency	Unique key in DB	Fast lookup / optional cache at larger scale.
Notifications	Synchronous simple UI update	Queue / event system later.
Analytics	Basic queries	Separate analytical pipeline later; do not burden the transaction path.
Architecture	Modular monolith	Extract services only when scale / team boundaries justify it.

RECOMMENDED, NOT MANDATED

16. Recommended Hackathon Tech Stack

Layer	Recommended	Reason
Frontend	React + Vite or Next.js	Fast UI development and familiar component model.
Backend	FastAPI / NestJS / Express - choose what the team knows best	Hackathon speed matters more than framework prestige.
Database	PostgreSQL	Strong ACID transactions, constraints and row locking.
ORM	Prisma / SQLAlchemy / TypeORM	Speeds schema and query development while allowing transactions.

Auth	Session or JWT + password hash	Simple and sufficient for closed prototype.
Deployment	Vercel for frontend + Render/Railway/Fly/any available backend host	Quick demo deployment; local fallback should also work.
Testing	Unit + integration tests for Payment Engine	Reliability scenarios should be automated, not only manually demoed.

Best architecture choice for 9 hours

Do not build microservices. A clean modular monolith with a strong Payment Engine is faster, easier to test and easier to defend.

EXECUTION PLAN

17. 9-Hour Build Plan

Time	Goal	Deliverable
0:00 - 0:30	Lock scope, schema, screens, responsibilities	No feature debate after this point.
0:30 - 1:30	Project setup + auth + DB migration	Users can register/login; account is created.
1:30 - 3:15	Payment Engine	Atomic send, validation, idempotency, history.
3:15 - 4:15	Money Request	Create, accept, reject; reuse Payment Engine.
4:15 - 5:15	Concurrency + reliability tests	Double-click, retry, insufficient balance, simultaneous spend.
5:15 - 6:15	Safe Send + Trust Receipt	Visible innovation on top of stable core.
6:15 - 7:00	Trust Lab	Judge-facing reliability demo.
7:00 - 7:45	UI polish + loading/error states	Clear user experience.
7:45 - 8:20	Deployment + backup local demo	Hosted URL and offline fallback.
8:20 - 9:00	Pitch rehearsal + bug freeze	Only critical fixes; prepare scripted demo.

17.1 Team split suggestion

Role	Primary ownership
Backend / Financial Logic	Payment Engine, ledger, DB transactions, idempotency, concurrency tests.

Frontend / UX	Dashboard, Send, Request, Safe Send, receipts, error states.
Integration / QA / Pitch	Auth integration, Trust Lab, seed data, deployment, demo script and testing.

PRESENTATION STRATEGY

18. Demo Script and Pitch

18.1 90-second story

15. Open with the problem: “Sending money is easy. Keeping it correct when users double-click, networks retry and two actions happen at the same time is the real challenge.”
16. Show normal Send Money: BDT 2,500 from Hasan to Saif.
17. Show the Trust Receipt and exact balance change.
18. Open Trust Lab and fire the Duplicate Click Test: 5 requests, 1 financial effect.
19. Run Concurrent Spend Test: BDT 1,000 balance; two BDT 700 transfers; one succeeds and one is rejected.
20. Show Ledger Integrity: difference BDT 0.
21. End with Safe Send or Smart Collect as the user-facing innovation.

Pitch line

“TrustFlow is not another wallet UI. It is a trust-first money movement system designed to remain correct under accidental duplicate actions, concurrent transfers, network retries and human mistakes.”

18.2 What not to lead with

- Dark mode
- QR code
- Animation
- Chatbot
- A long list of incomplete features

These can be nice extras, but they are weaker differentiators than proven financial correctness.

PRESENTATION PREPARATION

19. Judge Q&A Preparation

Likely question	Strong short answer
What is innovative?	Safe Send + Trust Lab on top of a failure-safe payment engine. We prove reliability instead of only claiming it.
How do you prevent double payment?	Each logical request has a unique idempotency key; retries return the original result.
What if two transfers happen at once?	Balance check and deduction run inside one DB transaction with account locking / atomic update.

What if server fails after deducting?	Debit and credit are in the same database transaction, so failure rolls back both.
Why a ledger if you already have balance?	Balance is fast state; ledger is the immutable explanation and reconciliation source for every movement.
Why not microservices?	Nine-hour scope. A modular monolith is simpler, safer to integrate and still cleanly separates financial logic.
Where is AI?	AI is not necessary for the core problem. We use explainable rules; optional NLP can create a draft request but never execute money without confirmation.
How will it scale?	Stateless API, indexed/paginated transaction access and PostgreSQL first; services/partitioning only when real load requires them.
Is this real money?	No. The challenge explicitly allows a closed simulated money ecosystem.

FINAL CHECKLIST

20. Final Submission Checklist

Status	Item
☐	Working registration / login.
☐	Each user receives a wallet/account and simulated starting balance.
☐	Send Money works end-to-end.
☐	Request Money works end-to-end with accept/reject.
☐	Transaction history is visible.
☐	Backend rejects invalid, zero, negative, self and insufficient-balance transfers.
☐	Transfer is atomic: no partial debit/credit.
☐	Duplicate requests do not charge twice.
☐	Concurrent spend test passes.
☐	Transaction / ledger IDs are traceable.
☐	At least one user-facing innovation works: Safe Send or Smart Collect.
☐	Trust Lab or equivalent reliability demo is ready.
☐	System runs with no real bank/payment integration.
☐	Team can explain every major engineering decision.
☐	Demo has seeded users and a fallback local environment.



Pitch is rehearsed and feature scope is frozen.

Recommended final scope

P0: Auth + Wallet + Send + Request + History + Atomicity + Duplicate Shield + Concurrency Protection.

P1: Safe Send + Trust Receipt + Trust Lab. Only add P2 features if all P0 tests are green.

Source-derived requirements in this blueprint are based on the 2-page PSTU IT Carnival 2026 Hackathon Challenge document supplied by the user. Proposed enhancements are explicitly marked as solution ideas, not original challenge requirements.